

Travaux pratiques

Programmation web II

TP 6

2024-2025

1) Travail Pratique 2 : Ma Première Application React - Liste de Tâches Statique

Objectifs du TP :

Mettre en pratique les concepts fondamentaux de React (composants fonctionnels, JSX, props, composition) en créant une application simple affichant une liste de tâches prédéfinies.

Contexte :

Vous allez développer une petite application web statique qui présente une liste de tâches. Chaque tâche possède un nom et un statut indiquant si elle est complétée ou non. L'ensemble de l'application sera construit en utilisant **exclusivement des composants fonctionnels**.

Tâches à réaliser :

1. Structure du Projet :

- Assurez-vous d'avoir un projet React fonctionnel (par exemple, créé avec create-react-app).
- Créez un répertoire src/components pour organiser vos composants personnalisés.

2. Composant TaskItem (Élément de Tâche Individuel) :

- Créez un composant nommé TaskItem dans le répertoire components.
- **Implémentation** : Développez ce composant **en tant que composant fonctionnel (function TaskItem(...))**.
- Ce composant doit recevoir deux informations via les props : le nom de la tâche (une chaîne de caractères) et son statut (un booléen, true pour complétée, false sinon).
- Le composant doit afficher le nom de la tâche. À côté du nom, il doit indiquer "(Terminée)" si la tâche est complétée, ou "(À faire)" sinon. Utilisez le JSX et une expression JavaScript (comme un opérateur ternaire) pour gérer cet affichage conditionnel.

3. Composant TaskList (Liste des Tâches) :

- Créez un composant nommé TaskList dans le répertoire components.
- **Implémentation** : Développez ce composant **en tant que composant fonctionnel**.
- Définissez un tableau JavaScript directement dans ce composant contenant au moins 3 objets "tâche". Chaque objet doit avoir une structure similaire à { id: 1, name: "Nom de la tâche", completed: false }.

```
const tasks = [  
  { id: 1, name: "Comprendre les Composants", completed: true },  
  { id: 2, name: "Maîtriser les Props", completed: false },  
  { id: 3, name: "Explorer JSX", completed: false },  
];
```

- Utilisez le composant TaskItem que vous avez créé précédemment.
- Dans le JSX retourné par TaskList, utilisez la méthode .map() sur votre tableau de tâches pour afficher une liste () contenant une instance de TaskItem pour chaque tâche.
- Assurez-vous de passer correctement les props (nom et statut) de chaque tâche à l'instance correspondante de TaskItem.
- **Important** : N'oubliez pas d'ajouter la prop key requise par React lors du rendu de listes (utilisez l'id de la tâche).

4. Intégration dans App.js :

- Modifiez le composant principal App.js (qui devrait déjà être un composant fonctionnel).
- Importez et affichez le composant TaskList à l'intérieur du return de App. Vous pouvez ajouter un titre général (ex: <h1>Ma Liste de Tâches</h1>) dans App.js au-dessus de TaskList.

2) Travail Pratique 2 : Composants Réutilisables et Props - Affichage de Produits

Objectif :

Renforcer la compréhension de la création de composants fonctionnels réutilisables, de la composition de composants, et de l'utilisation des props pour passer des données dynamiques. Utiliser le rendu conditionnel simple basé sur les props.

Contexte :

Vous allez créer une section simple d'une page e-commerce qui affiche plusieurs produits. Chaque produit aura un nom, un prix et une image. Certains produits pourraient être marqués comme étant en promotion.

Tâches à réaliser :

1. Structure du Projet :

- Continuez dans votre projet React existant ou créez-en un nouveau.
- Assurez-vous d'avoir un répertoire `src/components`.
- Créez un sous-répertoire `public/images` (s'il n'existe pas déjà) et placez-y 5 images de produits données avec l'énoncé de ce TP

2. Composant `ProductDisplay` (Affichage d'un Produit) :

- Créez un **composant fonctionnel** nommé `ProductDisplay` dans `src/components`.
- Ce composant doit accepter les props suivantes :
 - `productName` (chaîne de caractères)
 - `price` (nombre ou chaîne de caractères)
 - `imageUrl` (chaîne de caractères - le chemin vers l'image dans le dossier public, ex: `/images/produit1.jpg`)
 - `isOnSale` (booléen - true si le produit est en promotion, false sinon)
- Dans le JSX retourné par `ProductDisplay` :
 - Affichez l'image du produit en utilisant la prop `imageUrl`. N'oubliez pas l'attribut `alt`.
 - Affichez le nom du produit (`productName`).
 - Affichez le prix (`price`).
 - **Rendu Conditionnel** : Utilisez une expression JavaScript (opérateur ternaire recommandé) pour afficher le texte " (En Promotion !)" à côté du prix *uniquement si* la prop `isOnSale` est true.

3. Composant `ProductList` (Liste des Produits) :

- Créez un **composant fonctionnel** nommé `ProductList` dans `src/components`.
- Importez le composant `ProductDisplay`.
- Dans le return de `ProductList`, utilisez la méthode `.map()` sur votre tableau de produits pour générer une liste de composants `ProductDisplay`.
- Pour chaque instance de `ProductDisplay`, passez des valeurs différentes pour les props `productName`, `price`, `imageUrl`, et `isOnSale`. Assurez-vous qu'au moins un produit a `isOnSale` à true et un autre à false pour tester le rendu conditionnel.

4. Intégration dans `App.js` :

- Modifiez le composant principal `App.js` (composant fonctionnel).
- Importez et affichez le composant `ProductList`. Vous pouvez ajouter un titre comme `<h1>Nos Produits</h1>` au-dessus.